

개인 포트폴리오 · 학생 시절 작품

던전앤파이터 포트폴리오

Win32 API · Direct2D 로 구현한 액션 RPG 프로토타입

맵 에디터 툴 개발부터 클라이언트까지 · 1인 개발

박 찬 욱

2017 · 한 달 목표로 시작한 미완성 프로토타입

이 문서는 완성작 소개가 아니다. "인기 온라인 게임을 한 달 동안 얼마나 재현할 수 있는가"라는 자기 과제로 시작한 개인 프로젝트이며, 전투 · 던전 루프는 상용 게임 수준의 판정으로 만들었지만 스킬 · 상점 · 캐릭터 생성 등 다수 시스템은 미구현 상태로 남아 있다. 6장에 그 목록을 정리했다.

1. 개요

넥슨의 온라인 액션 RPG '던전앤파이터'의 전투 · 던전 루프를 Win32 API와 Direct2D만으로 모작한 개인 프로젝트다. 완성보다 핵심 재미를 먼저 검증하는 쪽을 택해, 한 달이라는 스스로 정한 기간 동안 전투 시스템을 가장 깊게 만드는 데 시간을 몰아썼다. 그 결과 자동공격 3연타와 스킬 일부는 프레임 단위 타격 판정 · 크리티컬 · 넉업 · 보스 연출까지 상용 게임에 가까운 완성도로 동작하지만, 스킬 5종 중 3종, 상점, 캐릭터 생성 화면 등은 손대지 못한 채 스텝으로 남았다. 클라이언트뿐 아니라 맵 · 던전 · 게이트 · 몬스터 스폰을 직접 저작하는 MFC 기반 맵 에디터도 같은 소스를 공유하며 함께 만들었다.



다수의 몬스터를 상대하는 실제 플레이 화면. 화면에 뜬 1000 · 1500 은 자동공격 1타 · 2타에 설정된 데미지 값이며, 상단은 최근 피격 몬스터의 HP 바다.

개발 환경	Visual Studio 2015 (v140) · Win32 Desktop Application · C++ · 1인 개발 (클라이언트 + 맵 에디터 툴)
사용 기술	Win32 API, Direct2D(D2D1 HwndRenderTarget), FMOD Ex(사운드), MFC(에디터 UI). 외부 게임 엔진 없음
개발 기간	2017년, 한 달을 목표로 시작한 프로토타입 — 완성이 아니라 기간 내 도달 범위를 확인하는 것이 목표
소스 규모	클라이언트 · 툴이 공유하는 소스 100여 개 파일. 전투 · 애니메이션 · 던전 · 인벤토리 · 맵 에디터 포함
구현 범위	자동공격 3연타, 스킬 2종(정상 동작), 몬스터 · 보스 전투, 던전 방 전환과 게이트, 인벤토리 · 장비 · 아이템 획득, 맵 에디터(타일 · 오브젝트 · 게이트 · 몬스터 스폰 · 던전 조립 · 미니맵)

2. 전투 시스템 — 프레임에 히트박스를 붙인다

이 프로젝트에서 가장 공들인 부분이다. 히트박스를 '무기가 지나가는 동안 매 프레임 검사'하는 대신, 애니메이션의 특정 프레임 하나하나에 히트박스를 직접 등록해 두는 방식으로 만들었다.

AddHitCollider(frame, offset, size, damage, isFloat, vPower, hPower)가 프레임 번호를 키로 하는 hash_map<int, Collider*>에 콜라이더를 등록하고, 매 틱 현재 재생 프레임에 등록된 콜라이더가 있는지만 확인한다. 등록된 프레임이 아니면 그 틱에는 판정 자체가 존재하지 않는다 — '몇 프레임째에 칼끝이 닿는가'를 애니메이터가 정하듯 코드로 정하는 구조다.

자동공격 3타 애니메이션 — 히트박스는 35프레임에만 존재한다



```
// Player.cpp
m_stateMachine->AddState(_T("auto_attack_3"),
    _T("character_body_auto_attack_3"), STATE_FLAG_PLAY_ONCE);
m_stateMachine->GetStateAnimation(_T("auto_attack_3"))
    ->AddHitCollider(35, {50, 20}, {100, 50}, 2000, true, 470.f, 72.f);
```

자동공격 1타 · 2타 · 3타는 각각 3 · 12 · 35 프레임에 히트박스를 갖고, 데미지는 1000 · 1500 · 2000이다. 3타에만 isFloat 이 켜져 있어 마지막 타격에서 몬스터가 떠오른다.

같은 프레임이 여러 틱에 걸쳐 유지될 수 있으므로, 한 번의 타격이 중복으로 여러 번 들어가지 않도록 콜라이더마다 이미 맞춘 대상을 담아두는 목록을 두고 프레임이 바뀔 때마다 비운다. 실제 적중은 아래 네 검사를 모두 통과해야 성립한다.

적중 판정 파이프라인 — 네 검사를 모두 통과해야 데미지가 들어간다



데미지 · 넉업 파이프라인

몬스터는 매 틱 자신의 몸통 콜라이더를 플레이어의 현재 타격 콜라이더 및 활성 스킬 콜라이더와 검사한다. 적중하면 0~100 난수로 크리티컬(치명타 확률 이하 시 1.5배)을 굴리고, HP를 깎은 뒤 크리티컬 여부에 따라 색이 다른 데미지 숫자를 띄우고, HP바에 즉시 줄지 않고 서서히 따라 줄어드는 흰색 '소프트 게이지'를 엮는다. 콜라이더가 isFloat이면 저장된 수직 · 수평 힘으로 몬스터를 띄우고 상태를 float로 전환, 이후 중력 가속도(780)가 누적되며 자연스럽게 낙하해 down(다운) 상태로 이어진다.

방어력(physicalDef/magicalDef) 스탯은 존재하지만 이 파이프라인 어디서도 읽지 않는다 — 화면에는 표시되지만 실제 데미지 감소는 없다.

스킬과 이펙트

스킬은 GameObject를 상속한 별도 오브젝트로, 자체 애니메이션 목록을 런타임에 생성한다. 화염 계열 스킬은 0.25초 간격으로 화염 · 연기 애니메이션 쌍을 정해진 간격만큼 밀어가며 4회 생성해, 불길의 캐릭터가 바라보는 방향으로 뻗어나가도록 구현했다. 몬스터가 죽을 때 8~10개의 '고기' 조각이 흩어지는 연출, 보스 처치 시 전역 프레임레이트를 5로 낮춰 만드는 슬로모션 사망 시퀀스도 이 시기에 넣은 것이다.



화염 계열 스킬이 발동한 순간. 스킬 콜라이더도 자동공격과 완전히 같은 판정 경로를 통과하므로, 판정 코드를 스킬마다 새로 쓰지 않아도 된다.

3. 상태 머신 — 문자열 키로 잇는 커스텀 FSM

플레이어와 몬스터는 모두 직접 만든 유한 상태 머신으로 움직인다. 하나의 상태는 애니메이션과 전이 목록을 갖고, 전이는 이름 붙은 bool 값(예: is_input_attack_key)이나 float 값 비교, 혹은 '이 애니메이션이 한 번 다 재생됐는가' 조건으로 정의된다. 매 틱 현재 상태의 전이 목록을 순서대로 검사해 처음 만족하는 조건으로 상태를 바꾼다.



플레이어는 idle · combat · walk · auto_attack_1/2/3 · jump · fall · upslash · farming · fire_wave · ice_wave · lightning_wave 상태를, 몬스터는 idle · walk · hit · down · float · combat · attack 상태를 갖는다. 장비 파츠는 독립된 애니메이션 상태를 갖지 않고, 렌더할 때마다 몸통 애니메이션의 현재 프레임으로 강제 동기화해 장비가 몸에서 어긋나지 않도록 했다.

카메라

화면 중앙 근처에 데드존 사각형을 두고, 플레이어가 그 밖으로 나갈 때만 카메라를 움직인다. 가로 이동은 매 프레임 남은 거리를 60분의 1씩 줄이는 지수 감쇠로 부드럽게 따라가고, 세로 이동은 즉시 따라간다. 다만 전투 전용 카메라 연출 — 피격 시 흔들림, 보스전 줌인, 락온 — 은 전혀 없다. 전투 중과 평상시의 카메라 로직이 완전히 동일하다.

4. 던전 구조 · 게이트 전환 · 렌더 순서

하나의 던전은 방 번호를 키로 하는 해시맵으로 여러 맵(방)을 들고 있고, 시작 방 · 보스 방 · 미니맵 타일 그리드 같은 메타데이터를 자체 제작한 바이너리(.dun) 포맷으로 저장한다. 방 사이는 방향(상하좌우) · 목적지 방 번호 · 도착 좌표를 가진 게이트 오브젝트가 있다.

던전 방 전환 — 게이트는 별도 클리어 조건 없이 몬스터 수만 본다



화면에 그릴 오브젝트는 한 프레임에 하나의 리스트로 모아 정렬한다. 먼저 바닥 타일을 앞쪽으로 분리한 뒤, 바닥과 비바닥 구간을 각각 화면상 가장 아래쪽 Y좌표 기준 오름차순으로 정렬해 — 아래에 있는(카메라에 가까운) 오브젝트가 나중에, 즉 위에 그려지도록 한다. 2D 횡스크롤 게임에서 캐릭터가 배경 앞뒤로 자연스럽게 겹치도록 만드는 전형적인 방식이다.

아이템 · 인벤토리

전투 외 시스템 중에서는 인벤토리 · 장비 쪽이 가장 온전히 동작한다. 30칸 슬롯을 장비 · 소비 · 재료 · 퀘스트 · 아바타 탭으로 나눠 쓰고, 드래그 앤 드롭으로 옮기며, 부위별 장착 결과가 캐릭터 렌더와 능력치에 함께 반영된다. 필드에 떨어진 아이템과 골드를 줍는 동작, 소비 아이템의 시간에 걸친 HP/MP 회복도 들어가 있다.



인벤토리 · 아바타 장착 창과 아이템 톨팁. 장착 시 힘 · 지능 · 체력 · 정신 보정이 실제로 적용된다.

구조적으로 아쉬운 부분도 있다. 던전 클리어 보상은 던전 종류와 무관하게 고정된 아바타 아이템 5종과 9000~12000 사이 난수 골드로 고정돼 있다 — 원본 주석에도 '임시'라고 명시된 자리표시자다. 방 · 맵 데이터를 저장하는 바이너리 포맷도 고정 크기 구조체를 그대로 파일에 쓰고 읽는 방식이라 버전 관리나 범위 검증이 없다.

5. 맵 에디터 — 클라이언트와 소스를 공유하는 구조

맵 에디터(DNF Tool)는 MFC Document/View로 만든 별도 실행 파일이지만, 맵 · 게이트 · 몬스터 · 충돌 · 상태 머신 등 게임 로직 소스 자체를 클라이언트와 그대로 공유한다. 같은 .cpp 파일이 전처리기 플래그에 따라 클라이언트용 바이너리와 에디터용 바이너리 양쪽으로 컴파일되며, 게이트를 예로 들면 클라이언트 빌드에서는 몬스터 전멸 여부로 열림 · 닫힘 스프라이트를 그리지만 에디터 빌드에서는 같은 자리에 인덱스 · 좌표 디버그 텍스트를 그린다. 덕분에 에디터에서 배치한 결과와 실제 플레이 중 동작이 항상 일치한다는 걸 별도로 검증할 필요가 없다.

클라이언트와 맵 에디터가 같은 소스를 공유하는 구조



타일 팔레트로 마을 맵을 저작하는 화면. 왼쪽은 맵 목록과 속성(크기 · 배경 · BGM · 컷신), 오른쪽은 타일 목록과 미리보기다. 맵 위의 초록색 영역은 다른 맵으로 이어지는 이동 지점으로, 목적지 맵 이름과 좌표가 함께 표시된다.

에디터에서 실제로 저작하는 것: 타일 팔레트(썸네일 미리보기 포함), 맵 오브젝트 · 콜라이더, 게이트(방향 · 목적지 맵 · 도착 좌표를 입력하는 다이얼로그가 곧 4장에서 설명한 게이트 파라미터 그대로다), 몬스터 · NPC 스폰(보스 체크박스 포함), 맵 속성(배경 · BGM · 컷신 여부), 던전 방 조립과 보스 방 지정, 미니맵 타일, 배경음악 선택, 아이템 데이터베이스. 다만 스프라이트별 프레임 기준점(.flg 파일)을 직접 편집하는 다이얼로그는 소스에서 찾지 못했다 — 이 부분은 에디터 밖에서 별도 작업으로 만든 것으로 보이며, 에디터 기능의 공백으로 남겨둔다.



같은 에디터에서 몬스터 · NPC를 배치한 화면. 배치된 개체는 실제 게임과 같은 스프라이트로 그려지고, 선택 영역과 발밑 충돌선이 함께 표시된다.



에디터로 배치한 NPC가 실제 게임에서 대사 창과 함께 동작하는 모습.

6. 미완성 요소 — 있는 그대로

한 달이라는 목표 기간 안에서 전투를 우선했기 때문에, 아래 항목들은 실제로 동작하지 않거나 손대지 못한 채 남아 있다.

- 스킬 5종 중 라이트닝 웨이브 · 소드 게이트 · 버프 3종은 빈 함수 — 키를 누르면 대기 모션은 재생되지만 실제 효과는 없다.
- 상점 · 스킬트리 · 캐릭터 생성/선택 화면은 UI 뼈대만 있거나(캐릭터 생성/선택) 완전히 빈 클래스(상점, 스킬트리)다. 게임은 하드코딩된 캐릭터로 곧장 시작한다.
- 방어력(물리/마법) 스탯은 화면에 표시되지만 데미지 계산에는 전혀 반영되지 않는다.
- 콤보에는 입력 유예(버퍼) 창이 없다 — 판정 순간 공격키 입력 여부만 그대로 검사한다.
- 피격 시 화면 흔들림, 보스전 카메라 줌인 등 전투 전용 카메라 연출이 없다.
- 던전 클리어 보상이 던전 종류와 무관하게 고정된 임시 드랍 테이블이다.
- 800x600 해상도가 카메라 · UI 좌표 계산 곳곳에 상수로 박혀 있어 해상도 독립적이지 않다.
- 맵 · 던전 저장 포맷이 고정 크기 구조체의 직접 파일 입출력이라 버전 관리와 범위 검증이 없다.

7. 2026년 복원 작업

이 문서를 쓰기 위해 2026년에 2017년 원본 소스를 9년 만에 다시 열어 최신 툴셋으로 재빌드했다. 이 장은 그 과정에서 마주친 문제와 고친 버그를 정리한 것으로, 앞선 장에서 인용한 코드와 화면은 이 재빌드본 기준이며 2017년 원본 그대로는 아니다.

- 플랫폼 툴셋을 v140/v141에서 v145로, Windows SDK를 최신 버전으로 리타겟했다.
- 최신 툴셋에서 삭제된 `<hash_map>`을 `std::unordered_map`으로 세임 처리했다.
- 소스가 CP949로 저장돼 있어, 편집 도구가 UTF-8로 잘못 저장하면 한글 주석·문자열이 깨진다는 것을 확인하고 CP949를 유지했다.
- Release 빌드에서 전체 프로그램 최적화(/GL)가 사전 컴파일 헤더와 충돌해 링크 오류가 나는 것을 찾아 옵션을 껐다.
- 툴과 클라이언트가 스프라이트 기준점 파일(.flg)을 배타적 모드로 열도록 돼 있어, 툴을 켜 채로 클라이언트를 실행하면 캐릭터가 안 보이는 버그를 발견하고 공유 읽기 모드로 고쳤다.

8. 회고

가장 남는 것은 전투부터 만들기로 한 순서 선택이었다. 애니메이션 프레임에 히트박스를 직접 붙이는 방식은 지금 봐도 과하지 않으면서 원작의 타격감에 가까운 결과를 냈고, 크리티컬·넉업·보스 연출까지 엮이면서 '한 달짜리 습작치고는'이라는 수식어를 뗄 만한 부분이라고 생각한다. 클라이언트와 에디터가 소스를 그대로 공유하도록 한 것도 좋은 선택이었다 — 에디터에서 배치한 게이트·몬스터가 게임에서 다르게 동작할 걱정을 애초에 없앴다.

반면 범위는 분명히 욕심을 냈다. 스킬 5종, 상점, 캐릭터 생성까지 손을 뻗어 놓고 한 달 안에 절반도 마무리하지 못했다. 다시 만든다면 전투·던전 한 사이클만 완주하는 범위로 처음부터 좁혔을 것 같고, 스킬 데이터도 코드에 박아 넣는 대신 파일로 분리해 반복 작업을 줄였을 것이다. 방어력처럼 화면에는 있지만 실제로 계산에 안 들어가는 '표시만 되는 스탯'을 남겨두지 않는 것도 다음에는 지키고 싶은 기준이다.